CENTRO UNIVERSITÁRIO DE MARINGÁ – UNICESUMAR
ENGENHARIA DE SOFTWARE | ANÁLISE E DESENVOLVIMENTO DE SISTEMAS
ATIVIDADE DE ESTUDO PROGRAMADA (AEP) – 4º SEMESTRE (2026.2)

PROJETO MOVECITY:
APLICAÇÃO WEB PARA MOBILIDADE URBANA SUSTENTÁVEL

Documento de Arquitetura e Requisitos – 1ª Entrega Bimestral (versão revisada)

Equipe de Desenvolvimento	RA
Integrante 1: Miguel Souza Colis	25014158-2
Integrante 2: Arthur Riki Farias Tanaka	25224963-2
Integrante 3: João Gustavo Zanoni Gonçalves	25357434-2

ODS vinculados: ODS 11 – Cidades e Comunidades Sustentáveis (principal) e ODS 13 – Ação contra a Mudança Global do Clima (complementar)

Repositório GitHub (público):

https://github.com/Arthurtanaka/AEP-ESOFT4S

Sumário

1. Descoberta
2. Concepção e Alinhamento aos ODS
3. Justificativa Técnica e Arquitetural
4. Ambiente de Controle de Versão
5. Estrutura do Repositório
6. Lista de Requisitos do Sistema
7. Diagrama de Classes (Modelagem Orientada a Objetos)
8. Diagrama de Banco de Dados (DER)
9. Cronograma de Execução
10. Conclusão

Referências

Maringá, novembro de 2026

1. DESCOBERTA

1.1 Contexto e problema identificado

O processo de descoberta partiu de uma pergunta central: por que se deslocar pela cidade costuma ser lento, imprevisível e ambientalmente custoso? A equipe pesquisou e levantou o problema dos congestionamentos e do transporte urbano pouco eficiente. Em muitas cidades, o excesso de carros particulares, a falta de integração entre os meios de transporte e a baixa qualidade do transporte público provocam engarrafamentos, aumento da poluição do ar, maior tempo de deslocamento e redução da qualidade de vida.

Esses problemas aparecem no dia a dia de quem precisa decidir como ir de um ponto a outro: a pessoa raramente dispõe de uma visão única e comparável das alternativas e, por isso, tende a repetir o hábito de usar o carro particular, mesmo quando existem opções mais rápidas, previsíveis ou sustentáveis.

1.2 Partes interessadas

- **Passageiro (usuário):** pessoa que precisa se deslocar e deseja escolher a melhor alternativa considerando tempo, distância e impacto ambiental.
- **Administrador do sistema:** responsável por manter atualizados o catálogo de rotas, os meios de transporte e os alertas que o sistema apresenta.
- **Cidade e sociedade:** beneficiárias indiretas, pois a redução do uso do transporte individual diminui o fluxo de veículos e a emissão de poluentes.

1.3 Dores, origens e requisitos que as respondem

A tabela a seguir relaciona cada dor identificada à sua origem, à parte interessada afetada e ao requisito da Seção 6 que a resolve, o que garante a rastreabilidade entre o problema e a solução proposta.

Dor identificada	Origem da dor	Parte interessada	Requisitos
Congestionamentos frequentes: aumentam o tempo de viagem e tornam os deslocamentos menos previsíveis.	Grande volume de veículos particulares nas vias e preferência pelo transporte individual.	Passageiro	RQ05, RQ06
Falta de integração entre transportes: dificulta combinar ônibus, bicicleta, caminhada e automóvel em uma mesma viagem.	Cada meio de transporte é consultado isoladamente, sem uma rota que combine vários meios.	Passageiro	RQ03, RQ05
Informações dispersas: o usuário precisa consultar diferentes fontes para decidir como se deslocar.	Dados sobre rotas, meios de transporte e ocorrências ficam espalhados em fontes diferentes.	Passageiro	RQ05, RQ06, RQ10

Dor identificada	Origem da dor	Parte interessada	Requisitos
Dependência do carro particular: contribui para o aumento do fluxo de veículos e da emissão de poluentes.	O usuário não enxerga o custo ambiental de cada escolha e, por isso, não tem estímulo para trocá-la.	Passageiro e cidade	RQ06, RQ07
Baixa previsibilidade: sem uma visão comparativa das alternativas, o usuário pode escolher uma rota menos eficiente.	Ausência de comparação lado a lado de tempo, distância e impacto ambiental.	Passageiro	RQ06, RQ10
Necessidade de simplicidade: a solução deve apresentar as informações de forma clara e fácil de utilizar.	Ferramentas com muitas telas e opções afastam quem só quer decidir rapidamente.	Passageiro	RQ05 e atributo de usabilidade (6.2)
Informações desatualizadas: rotas e avisos que não refletem a realidade levam a decisões ruins.	Falta de um responsável que mantenha o catálogo de rotas, transportes e alertas.	Administrador	RQ03, RQ04, RQ10

Tabela 1: Dores, origens, partes interessadas e requisitos relacionados.

1.4 Entrega de valor para as partes interessadas

- **Para o passageiro:** passa a ter, em uma única aplicação, informações para comparar rotas e meios de transporte, o que facilita decisões que considerem tempo, distância e sustentabilidade.
- **Para o administrador:** dispõe de um espaço único para manter rotas, meios de transporte e alertas, garantindo que o usuário veja informações atualizadas.
- **Para a cidade:** a proposta incentiva alternativas ao transporte individual e contribui para uma mobilidade mais eficiente e com menos emissões.

1.5 Regras de negócio identificadas

As regras abaixo definem como o sistema deve se comportar e serão implementadas na camada Service (Seção 3).

Código	Regra de negócio	Requisitos
RN01	Cada e-mail só pode ser usado em um único cadastro de usuário.	RQ01
RN02	A senha nunca é armazenada em texto puro; o sistema guarda somente o seu hash.	RQ01, RQ02
RN03	Somente usuários com perfil de administrador podem cadastrar, alterar ou excluir rotas, meios de transporte e alertas.	RQ03, RQ04, RQ10
RN04	Toda rota deve possuir ao menos um trecho; ao excluir uma rota, seus trechos também são excluídos.	RQ03

Código	Regra de negócio	Requisitos
RN05	O tempo total e o impacto ambiental total de uma alternativa são a soma dos valores de seus trechos, cada um calculado conforme o meio de transporte usado nele.	RQ06
RN06	A alternativa mais sustentável é a de menor impacto ambiental total; em caso de empate, vence a de menor tempo.	RQ07
RN07	A consulta de rotas é livre, mas somente consultas de usuários autenticados são registradas no histórico.	RQ05, RQ09
RN08	Um usuário não pode favoritar a mesma rota mais de uma vez.	RQ08
RN09	Alertas com status inativo não são exibidos ao usuário.	RQ10

Tabela 2: Regras de negócio do MoveCity.

2. CONCEPÇÃO E ALINHAMENTO AOS ODS

2.1 Descrição do projeto

O MoveCity é uma aplicação web de mobilidade urbana sustentável. O sistema permite que o usuário informe origem e destino, consulte as alternativas de deslocamento cadastradas e compare as opções de transporte, apresentando as informações de forma centralizada e destacando a alternativa mais sustentável. Cada alternativa é uma rota formada por um ou mais trechos, e cada trecho utiliza um meio de transporte (carro, ônibus ou bicicleta), o que permite representar viagens que combinam diferentes meios.

2.2 Alinhamento aos ODS

O projeto está alinhado principalmente ao **ODS 11 – Cidades e Comunidades Sustentáveis**, cujo propósito é tornar as cidades e os assentamentos humanos inclusivos, seguros, resilientes e sustentáveis, e, de forma complementar, ao **ODS 13 – Ação contra a Mudança Global do Clima**. A tabela mostra de que modo o sistema contribui com as metas do ODS 11 diretamente ligadas à mobilidade.

ODS / Meta	O que a meta propõe	Como o MoveCity contribui	Requisitos
ODS 11 – Meta 11.2	Até 2030, dar acesso a sistemas de transporte seguros, acessíveis e sustentáveis para todos, ampliando o uso do transporte público.	Reúne em uma só consulta alternativas que incluem ônibus (transporte coletivo) e bicicleta (transporte não motorizado), tornando essas opções visíveis e comparáveis com o carro e destacando a mais sustentável.	RQ05, RQ06, RQ07
ODS 11 – Meta 11.6	Até 2030, reduzir o impacto ambiental negativo per capita das cidades, com atenção especial à qualidade do ar.	Mostra o impacto ambiental estimado de cada alternativa (gramas de CO_2) e indica a de menor impacto, estimulando a troca do carro individual por opções que emitem menos poluentes.	RQ06, RQ07

ODS / Meta	O que a meta propõe	Como o MoveCity contribui	Requisitos
ODS 13 – Ação contra a Mudança Global do Clima (complementar)	Combater a mudança do clima e seus impactos, o que inclui reduzir emissões e ampliar a conscientização.	Ao exibir a emissão de cada escolha de deslocamento e manter o histórico de consultas, informa o usuário sobre o efeito climático de suas decisões diárias.	RQ06, RQ07, RQ09

Tabela 3: Relação entre as metas dos ODS e as funcionalidades do MoveCity.

2.3 Como o sistema soluciona o problema

O MoveCity atua sobre a forma como o usuário decide se deslocar. Ao concentrar em um único lugar as informações sobre rotas e meios de transporte, o sistema reduz a necessidade de consultas dispersas; ao comparar as alternativas por tempo, distância e impacto ambiental, deixa claro o custo de cada escolha; e ao destacar a opção mais sustentável, orienta o usuário para alternativas mais adequadas ao seu objetivo e menos poluentes. Em conjunto, essas ações incentivam a redução do uso do transporte individual.

É importante delimitar o alcance da proposta: o MoveCity não elimina os congestionamentos por si só, mas contribui para que os deslocamentos sejam escolhidos de forma mais informada e sustentável, o que, em escala, favorece uma mobilidade urbana mais eficiente.

2.4 Escopo da solução

- **Dentro do escopo:** cadastro e autenticação de usuários; cadastro de rotas, trechos e meios de transporte pelo administrador; consulta e comparação de alternativas; destaque da opção mais sustentável; favoritos; histórico; alertas.
- **Fora do escopo desta versão:** navegação por GPS em tempo real, integração com serviços externos de mapas ou de horários de transporte público e pagamento de passagens. Distâncias, velocidades e emissões são valores cadastrados e estimados.

3. JUSTIFICATIVA TÉCNICA E ARQUITETURAL

Cada escolha tecnológica a seguir é justificada pela forma como suporta a lista de requisitos e as regras de negócio definidas neste documento.

3.1 Linguagem de programação: Java

A linguagem Java foi escolhida por sua forte orientação a objetos, que suporta diretamente o que o sistema exige: herança (Usuario e Transporte possuem subclasses), polimorfismo (cada meio de transporte calcula tempo e impacto à sua maneira, o que sustenta RQ06 e RQ07), classes abstratas e composição (uma Rota é composta por Trechos). A tipagem estática permite detectar erros ainda na compilação, o que favorece a organização e a manutenção do código. Java também oferece o JDBC, API nativa para acesso a bancos de dados relacionais, e executa em qualquer sistema operacional que possua a máquina virtual Java (Windows, Linux e macOS), o que facilita a execução do projeto em diferentes ambientes.

3.2 Banco de dados: MySQL

O MySQL é um sistema gerenciador de banco de dados relacional amplamente utilizado, gratuito e multiplataforma, adequado ao modelo entidade-relacionamento do projeto. Os dados do MoveCity são naturalmente relacionais, com vários relacionamentos 1:N entre Usuário, Rota, Trecho, Transporte, Favorito, Histórico e Alerta. As chaves estrangeiras garantem a integridade desses vínculos; as restrições de unicidade implementam regras de negócio (RN01 e RN08); a exclusão em cascata implementa RN04; e as consultas com junção e soma permitem calcular distância, tempo e impacto de cada alternativa (RN05). A herança é representada com tabela única e uma coluna discriminadora (tipo), solução simples e suficiente para as hierarquias do projeto. O script de criação das tabelas ficará em /database/schema.sql.

3.3 Padrão arquitetural: arquitetura em camadas (Model, DAO, Service e Controller)

O projeto seguirá uma arquitetura em camadas, em que cada camada tem uma responsabilidade única:

- **Model:** entidades do domínio e aplicação dos conceitos de POO (herança, polimorfismo, composição e classes abstratas).
- **DAO / Repository:** persistência e comunicação com o banco de dados por meio do JDBC.
- **Service:** regras de negócio (RN01 a RN09), como validar o perfil do usuário, somar tempos e impactos e escolher a alternativa mais sustentável.
- **Controller:** camada de entrada que recebe as requisições do front-end, chama o Service e devolve a resposta, sem conter regra de negócio.

Essa separação favorece a organização do projeto e facilita a sua manutenção: as regras de negócio ficam concentradas em um só lugar, o banco pode ser alterado sem afetar essas regras, e os integrantes podem trabalhar em camadas diferentes ao mesmo tempo, o que permite a contribuição de todos no repositório.

3.4 Interface: HTML5, CSS3 e JavaScript

A interface web responsiva, feita com HTML5, CSS3 e JavaScript, comunica-se com o back-end por requisições HTTP com troca de dados em JSON. Por ser executada no navegador, atende ao atributo de usabilidade e de responsividade (Seção 6.2) sem exigir instalação pelo usuário.

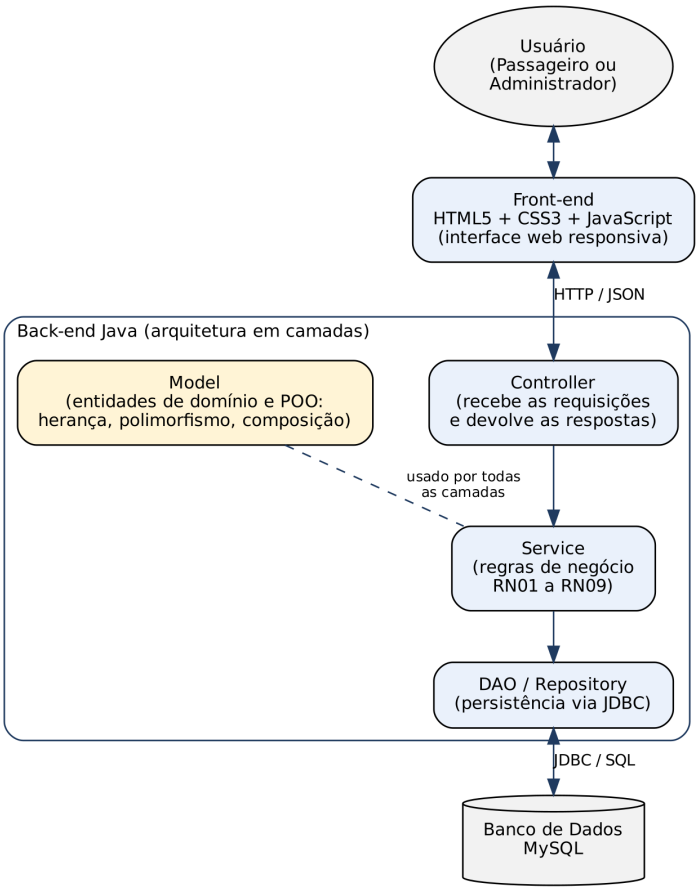


Figura 1: Visão geral da arquitetura em camadas do MoveCity.

4. AMBIENTE DE CONTROLE DE VERSÃO

Link do repositório do projeto no GitHub:

https://github.com/Arthurtanaka/AEP-ESOFT4S

O repositório é público, para permitir a consulta e a auditoria pelos professores sem necessidade de login ou convite. Ele contém a estrutura de diretórios do projeto, o arquivo README.md (que espelha a lista de requisitos e o cronograma deste documento), os diagramas de modelagem e o script SQL de criação do banco. O histórico de commits registra a contribuição de cada integrante da equipe.

5. ESTRUTURA DO REPOSITÓRIO

A organização das pastas no repositório segue a padronização oficial da atividade:

```
AEP-ESOFT4S/
├── src/                      Código-fonte da aplicação Java
│   ├── model/                Entidades de domínio (POO)
│   ├── dao/                  Acesso ao banco (JDBC)
│   ├── service/              Regras de negócio
│   ├── controller/           Entrada das requisições
│   └── util/                 Classes utilitárias
├── docs/                     Documentação do projeto
│   ├── AEP_MoveCity.pdf      Este documento
│   ├── diagrama_classes.png  Diagrama de classes
│   └── der.png               Diagrama entidade-relacionamento
├── database/
│   └── schema.sql            Script de criação das tabelas
├── README.md                 Descrição, requisitos e cronograma
└── .gitignore                Arquivos que não devem ser versionados
```

Item	Conteúdo
/src	Código-fonte da aplicação Java, organizado nos pacotes model, dao, service, controller e util.
/docs	Este documento em PDF e os diagramas de classes e de banco de dados.
/database	Script SQL de criação das tabelas (schema.sql).
README.md	Descrição do projeto, lista de requisitos, cronograma e instruções de execução.
.gitignore	Arquivos e diretórios que não devem ser versionados.

Tabela 4: Estrutura de diretórios do repositório.

6. LISTA DE REQUISITOS DO SISTEMA

6.1 Requisitos do sistema

Os requisitos abaixo foram identificados a partir do problema, das dores e das regras de negócio levantadas na Seção 1. A **entidade principal** do sistema, sobre a qual será desenvolvido o CRUD completo, é a **Rota** (RQ03).

Código	Descrição	Parte interessada
RQ01	O sistema deve permitir o cadastro de usuários, com nome completo, e-mail, senha e perfil (passageiro ou administrador) e, para passageiros, a preferência de deslocamento (por exemplo, mais rápido ou mais sustentável), não permitindo dois cadastros com o mesmo e-mail.	Passageiro e administrador
RQ02	O sistema deve permitir que o usuário cadastrado acesse o sistema informando e-mail e senha, liberando as funções de acordo com o seu perfil.	Passageiro e administrador

Código	Descrição	Parte interessada
RQ03	O sistema deve permitir ao administrador cadastrar, consultar, atualizar e excluir rotas, informando origem, destino, descrição e os trechos que as compõem (ordem, distância em km e meio de transporte de cada trecho).	Administrador
RQ04	O sistema deve permitir ao administrador cadastrar, consultar, atualizar e excluir meios de transporte, com nome, tipo (carro, ônibus ou bicicleta), velocidade média em km/h e emissão de CO_2 em gramas por km.	Administrador
RQ05	O sistema deve permitir ao usuário informar uma origem e um destino e apresentar as alternativas de rota cadastradas para esse par, com os meios de transporte de cada trecho.	Passageiro
RQ06	O sistema deve permitir comparar as alternativas lado a lado, exibindo para cada uma o tempo estimado em minutos, a distância total em km e o impacto ambiental estimado em gramas de CO_2.	Passageiro
RQ07	O sistema deve destacar a alternativa mais sustentável, isto é, a de menor impacto ambiental total, quando houver mais de uma alternativa disponível.	Passageiro e cidade
RQ08	O sistema deve permitir ao usuário autenticado salvar uma rota como favorita, listar suas rotas favoritas e removê-las.	Passageiro
RQ09	O sistema deve manter, para cada usuário autenticado, o histórico das consultas de rotas realizadas, registrando a rota consultada e a data e hora da consulta.	Passageiro
RQ10	O sistema deve permitir ao administrador registrar alertas vinculados a uma rota, com mensagem e status (ativo ou inativo), e exibir os alertas ativos ao usuário que consultar essa rota.	Administrador e passageiro

Tabela 5: Lista de requisitos do sistema MoveCity.

6.2 Atributos de qualidade esperados

- **Usabilidade:** interface simples, que permita consultar e comparar alternativas em poucas ações.
- **Responsividade:** a interface deve se adaptar a telas de computador, tablet e celular.
- **Segurança:** senhas armazenadas apenas como hash e funções administrativas restritas ao perfil de administrador.
- **Desempenho:** consulta e comparação de alternativas sem demora perceptível para o volume de rotas cadastradas.
- **Disponibilidade:** o sistema deve estar acessível sempre que o usuário precisar planejar um deslocamento.
- **Manutenibilidade:** código organizado em camadas, com convenções de nomenclatura (CamelCase) e comentários explicativos.

7. DIAGRAMA DE CLASSES (MODELAGEM ORIENTADA A OBJETOS)

O diagrama de classes (Figura 2) contempla os três conceitos de modelagem exigidos:

- **Herança:** Usuario (classe abstrata) é especializada em Passageiro e Administrador; Transporte (classe abstrata) é especializada em Carro, Onibus e Bicicleta.
- **Polimorfismo:** os métodos calcularTempo() e calcularImpacto() são declarados como abstratos em Transporte e sobrescritos (@Override) por Carro, Onibus e Bicicleta, cada um com a sua regra de cálculo; da mesma forma, autenticar() e getPerfil() são sobrescritos por Passageiro e Administrador. Assim, Trecho e Rota calculam tempo e impacto sem precisar saber qual é o meio de transporte concreto.
- **Composição de multiplicidade 1:N:** uma Rota é composta por 1..N Trechos; se a rota deixa de existir, seus trechos também deixam.

As demais associações são: Usuario 1:N Favorito, Usuario 1:N Historico, Administrador 1:N Alerta, Favorito, Historico e Alerta N:1 Rota e Trecho N:1 Transporte. Os atributos e métodos de cada classe correspondem às colunas e regras descritas nas seções anteriores.

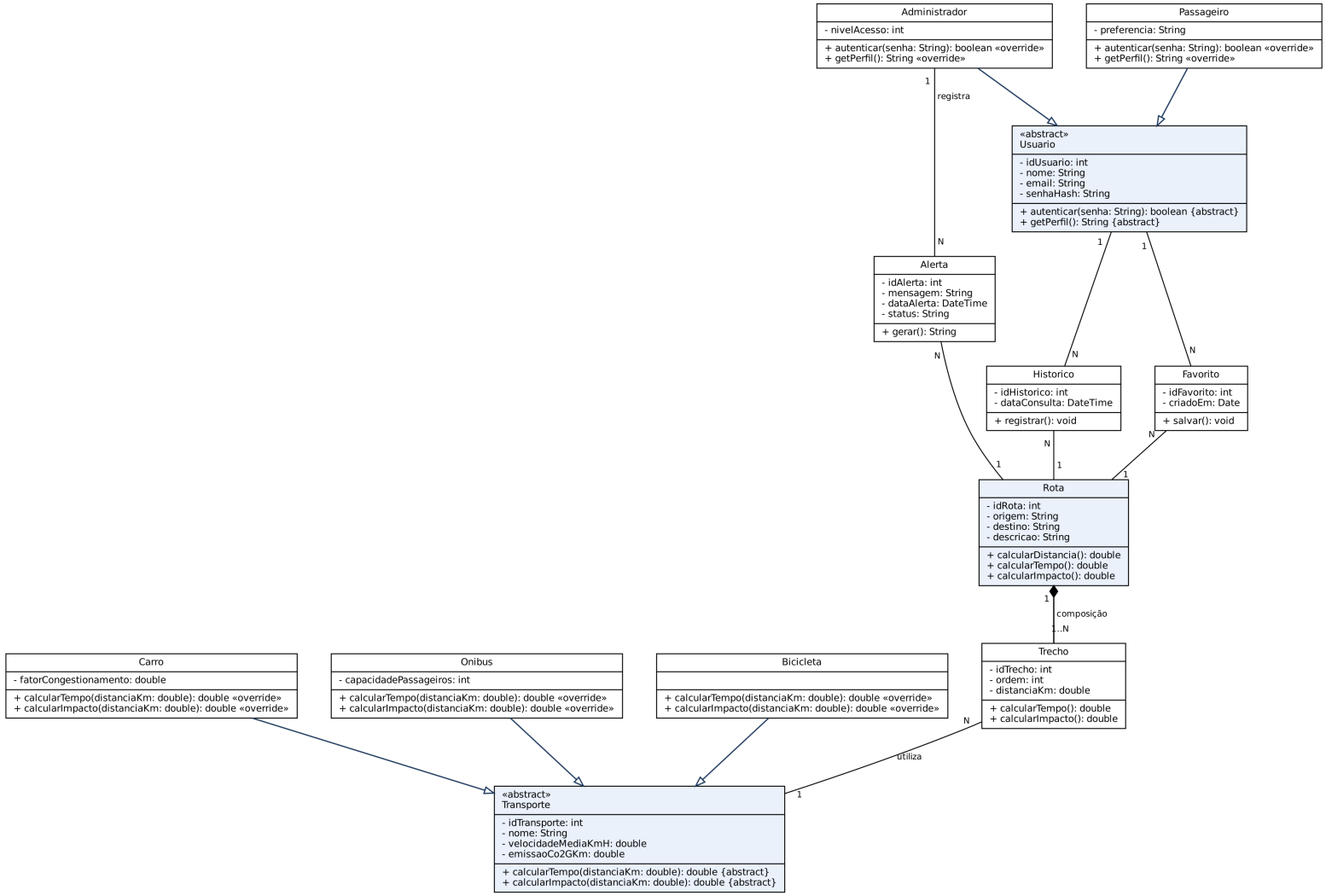


Figura 2: Diagrama de Classes do MoveCity (losango preenchido = composição; triângulo vazado = herança).

8. DIAGRAMA DE BANCO DE DADOS (DER)

O modelo relacional espelha as entidades do diagrama de classes, utilizando chaves primárias (PK) e estrangeiras (FK) para representar os relacionamentos entre usuários, rotas, trechos, meios de transporte, favoritos, históricos e alertas. Em particular:

- a hierarquia de Usuario (Passageiro e Administrador) é armazenada na tabela **usuario**, com a coluna tipo_usuario como discriminador;
- a hierarquia de Transporte (Carro, Onibus e Bicicleta) é armazenada na tabela **transporte**, com a coluna tipo como discriminador;
- a composição entre Rota e Trecho é representada pela chave estrangeira id_rota em **trecho**, com exclusão em cascata;
- Favorito e Historico ligam um usuário a uma rota, e Alerta liga uma rota ao administrador que o registrou.

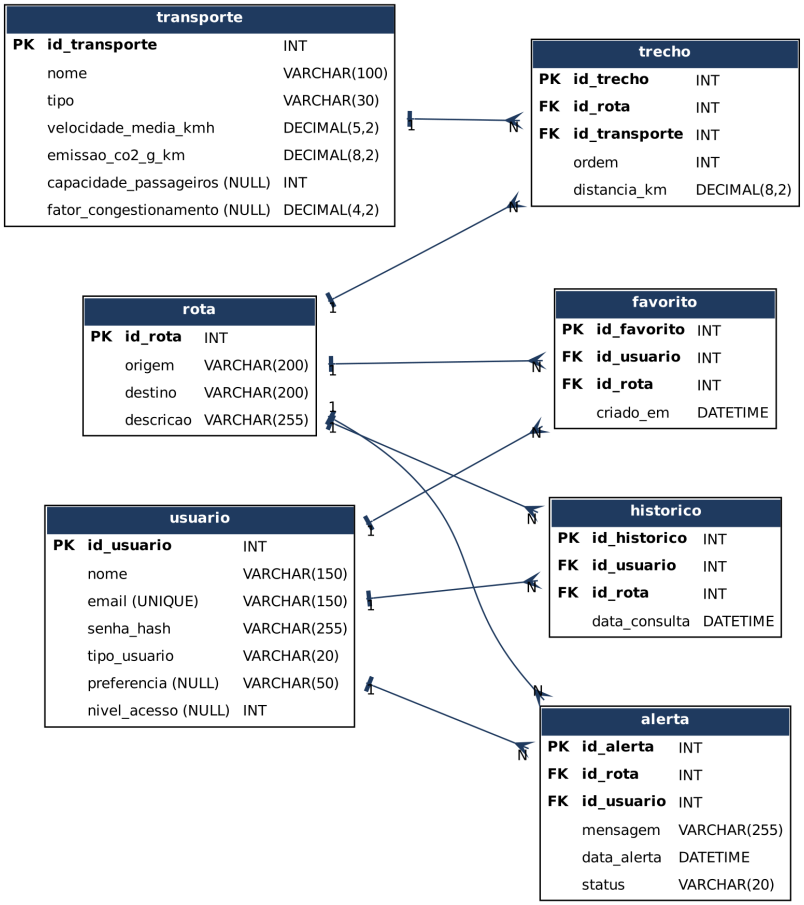


Figura 3: Diagrama Entidade-Relacionamento (DER). O script correspondente está em /database/schema.sql.

9. CRONOGRAMA DE EXECUÇÃO

O cronograma abrange as duas entregas da AEP: a Fase 1 corresponde ao prazo da 1ª entrega (de 27/08/2026 a 11/09/2026), com a revisão do documento conforme o retorno dos professores, e a Fase 2 corresponde ao desenvolvimento do software da 2ª entrega. A mesma tabela está espelhada no arquivo README.md do repositório.

Data	Atividade	Responsável
Fase 1 – Planejamento, escopo e estruturação (1ª entrega)		
27/08/2026	Levantamento de requisitos e definição do escopo	Todos os integrantes
31/08/2026	Modelagem do Diagrama de Classes e do DER	Miguel Souza Colis
03/09/2026	Criação e estruturação do repositório no GitHub	Arthur Riki Farias Tanaka
06/09/2026	Redação da Justificativa Técnica e Concepção do projeto	João Gustavo Zanoni Gonçalves
09/09/2026	Revisão geral do documento e formatação do PDF	Todos os integrantes
11/09/2026	Entrega final da 1ª etapa (PDF + link do GitHub)	Todos os integrantes
05/11/2026	Revisão e correção da 1ª entrega conforme o retorno dos professores (documento, README e repositório)	Todos os integrantes
Fase 2 – Implementação, rigor e integração (2ª entrega)		
09/11/2026	Publicação do repositório com estrutura de pastas, README e schema.sql atualizados	Arthur Riki Farias Tanaka
12/11/2026	Implementação do pacote model (Usuario, Passageiro, Administrador, Rota, Trecho, Transporte, Carro, Onibus, Bicicleta, Favorito, Historico e Alerta)	Miguel Souza Colis
16/11/2026	Implementação da camada DAO e da conexão JDBC com o MySQL	Arthur Riki Farias Tanaka
19/11/2026	Implementação da camada Service (regras RN01 a RN09) e do CRUD completo de Rota	João Gustavo Zanoni Gonçalves
23/11/2026	Implementação do Controller e da interface web	Miguel Souza Colis e João Gustavo Zanoni Gonçalves
26/11/2026	Testes de execução do sistema completo e instruções de execução no README	Todos os integrantes
30/11/2026	Entrega final da 2ª etapa (código-fonte e repositório)	Todos os integrantes

Tabela 6: Cronograma e divisão de responsabilidades.

10. CONCLUSÃO

O MoveCity foi concebido como uma solução tecnológica para o problema de congestionamentos e de transporte urbano pouco eficiente. Ao centralizar informações de rotas e alternativas de transporte, o sistema busca facilitar o planejamento dos deslocamentos, incentivar escolhas mais sustentáveis e contribuir para uma mobilidade urbana mais eficiente, em alinhamento às metas 11.2 e 11.6 do ODS 11 e, de forma complementar, ao ODS 13.

Como resultados desta etapa, a equipe definiu as dores e as regras de negócio do problema, uma lista de dez requisitos rastreáveis até essas dores, as escolhas tecnológicas (Java, MySQL e arquitetura em camadas) com a sua justificativa, o diagrama de classes com herança, polimorfismo e composição 1:N, o modelo de banco de dados correspondente, o cronograma de execução e a estrutura do repositório público no GitHub.

Como próximos passos, a 2ª entrega implementará o código-fonte completo do sistema, mantendo fidelidade a este planejamento: aplicação prática dos pilares da Programação Orientada a Objetos, CRUD completo da entidade principal (Rota), conexão ativa com o banco MySQL a partir do script de /database, instruções claras de execução no README e contribuição de todos os integrantes no repositório.

REFERÊNCIAS

GRUPO ASSESSOR DO SISTEMA ONU NO BRASIL SOBRE A AGENDA 2030. **Glossário de termos do Objetivo de Desenvolvimento Sustentável 11:** tornar as cidades e os assentamentos humanos inclusivos, seguros, resilientes e sustentáveis. Disponível em:

https://unhabitat.org/sites/default/files/documents/2019-04/glossario_-_ods_11.pdf

UNIVERSIDADE FEDERAL DE MINAS GERAIS (UFMG). **Cidades e comunidades sustentáveis.** Espaço do Conhecimento UFMG. Disponível em:

https://www.ufmg.br/espacodoconhecimento/cidades-e-comunidades-sustentaveis/